MODULE 08

STUDENT LABORATORY ACTIVITY MANUAL

CS0045 | Computer Graphics and Visual Computing

Lighting and Texture Mapping in C++ / OpenGL

Student: Ebron, Nicole Anne V. Section: TN35

Instructor: Dr. Hadji J. Tejuco Date: 11/10/2026

GitHub Repository: https://github.com/nicoleanneebron-art/Ebron-Nicole-Anne-V.

Learning outcomes

Explain and implement ambient, diffuse and specular lighting; configure materials, point and directional lights; apply texture coordinates and image textures to 3D objects; compare rendering results.

Development setup

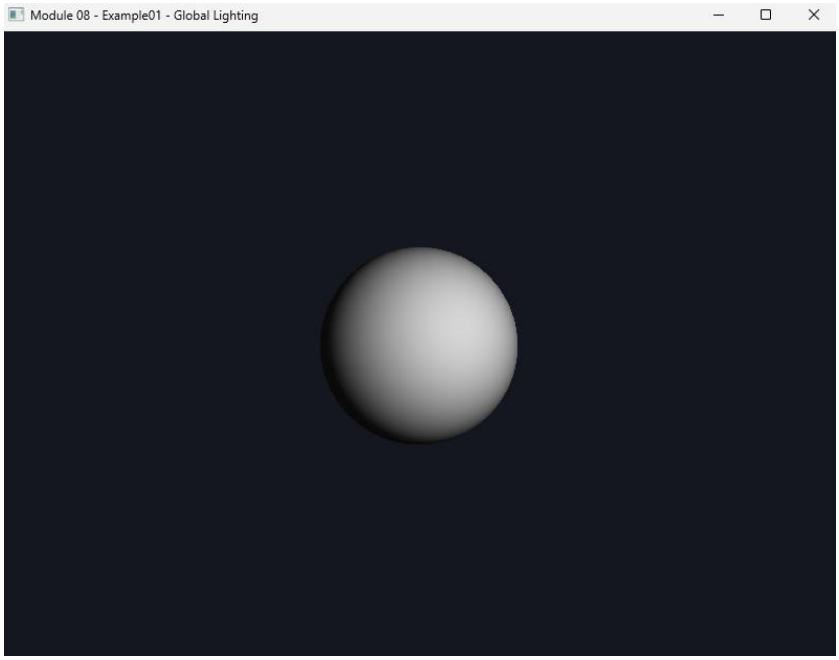
VS Code, MSYS2 UCRT64 GCC, GLEW and FreeGLUT. Texture-image activities additionally require stb_image.h (https://github.com/nothings/stb). Use a compatibility-profile OpenGL context because the module examples use fixed-function lighting and immediate-mode drawing.

Example build (MSYS2 UCRT64): g++ main.cpp -o lab.exe -lfreeglut -lglew32 -lopengl32 -lglu32

For texture images, keep the image file beside the executable or supply a correct path. Include #define STB_IMAGE_IMPLEMENTATION in exactly one .cpp file.

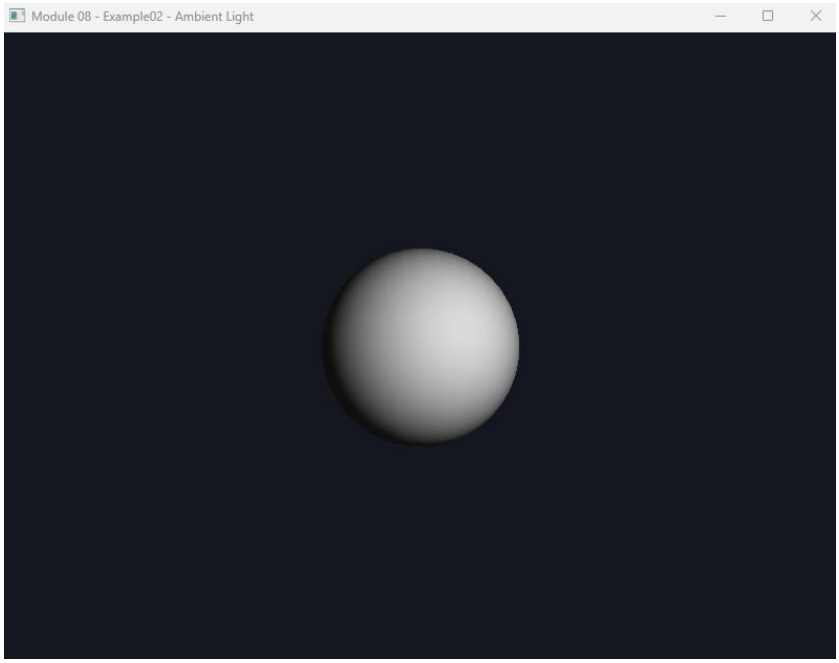
PART A — GUIDED EXAMPLES

Ex01 — Enable global lighting



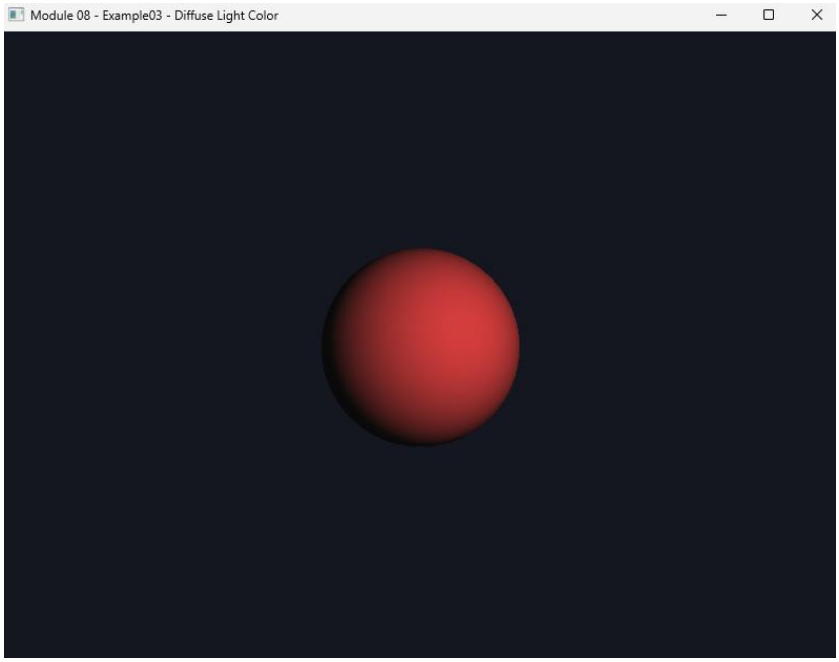
Example 01 - Enabling global lighting makes the sphere respond to OpenGL's light and material calculations. Its surface now shows shaded depth instead of a flat, uniform color.

Ex02 — Ambient light



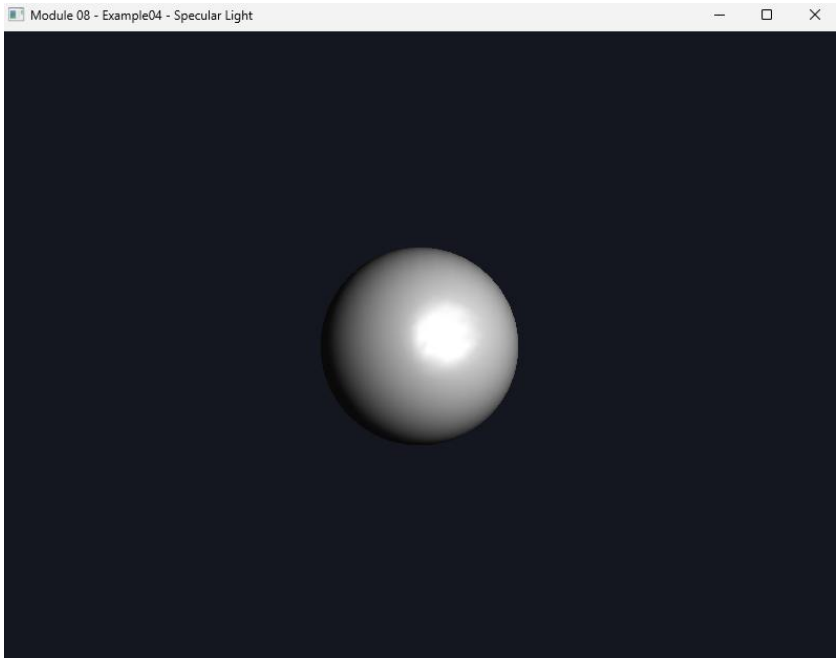
Example 02 - Increasing the global ambient light brightens areas that do not receive strong direct light. The darker side of the sphere becomes easier to see.

Ex03 — Diffuse light color



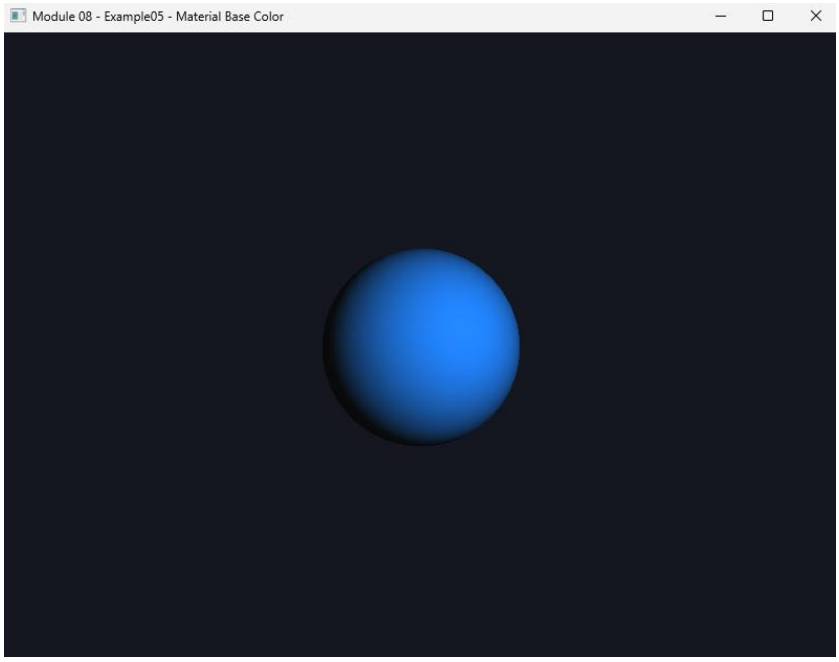
Example 03 - Changing GL_LIGHT0's diffuse color to red gives the directly illuminated surface a reddish tint. Shaded regions remain darker, preserving the sphere's form.

Ex04 — Specular light



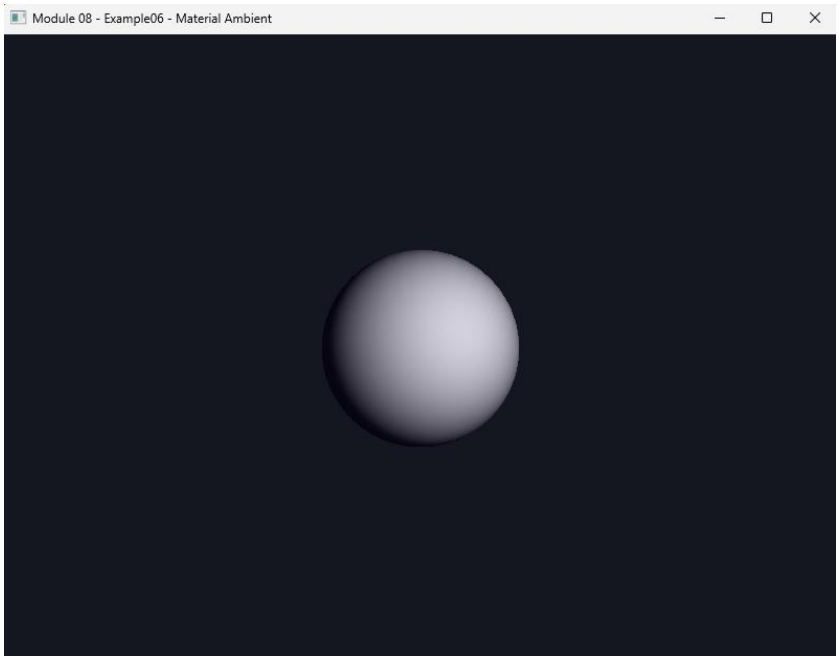
Example 04 - Adding white specular light and a reflective material creates a bright highlight on the sphere. The highlight indicates the area reflecting the light toward the viewer.

Ex05 — Material base color



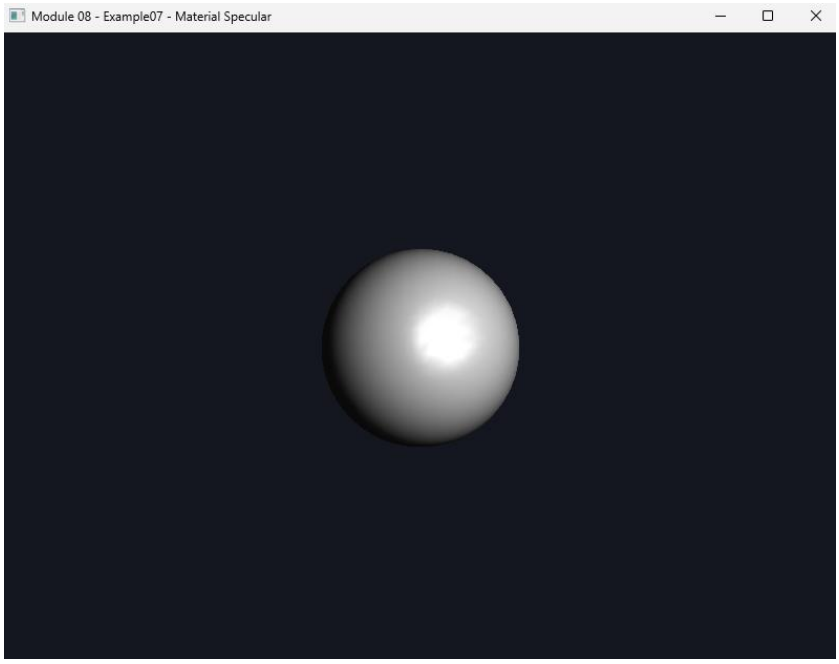
Example 05 - Setting the material's diffuse property changes the sphere's main illuminated color to blue. Lighting still controls how bright each part of the surface appears.

Ex06 — Material ambient



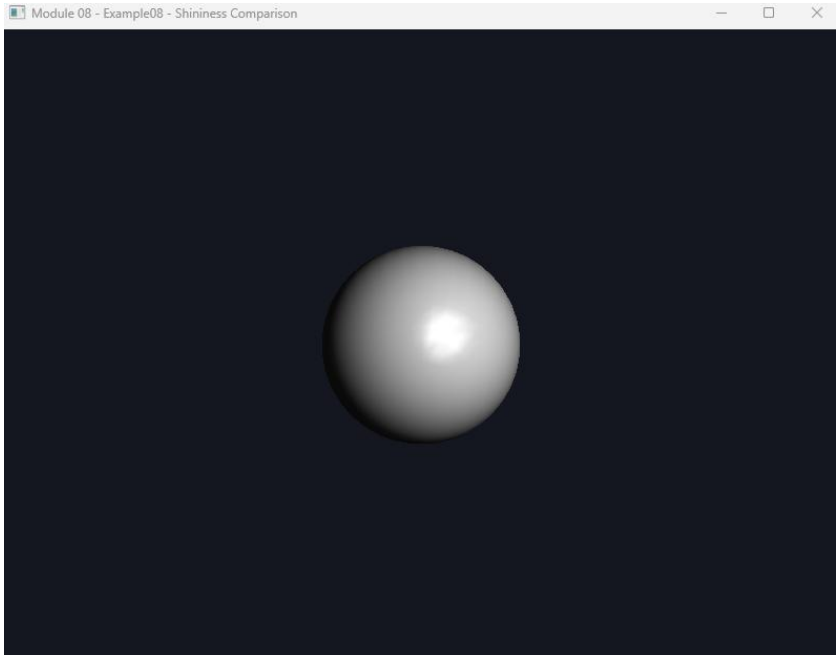
Example 06 - A purple ambient material changes the color visible in low-light areas. The result is most noticeable on the side facing away from the direct light.

Ex07 — Material specular



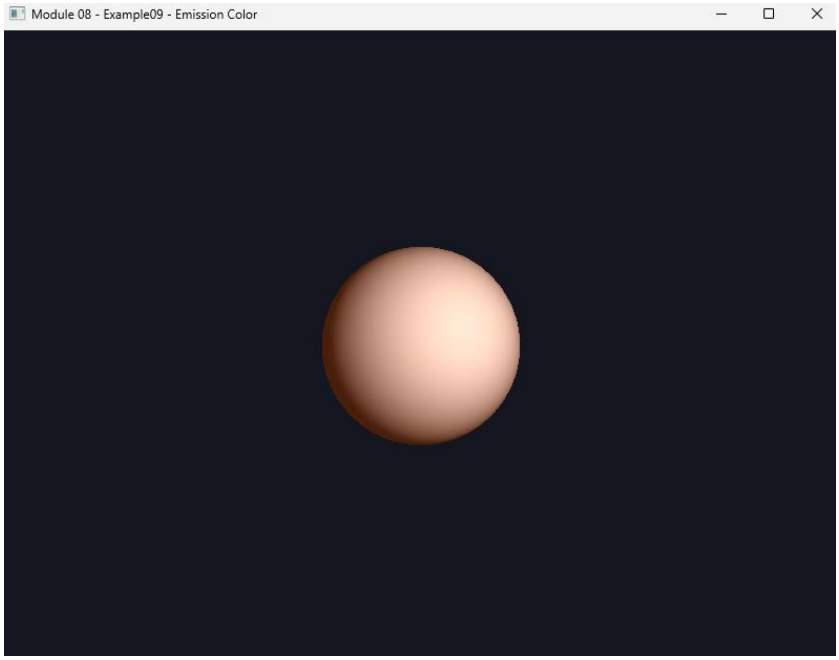
Example 07 - Giving the material a white specular component allows it to reflect a visible white highlight. The sphere appears smoother and more polished.

Ex08 — Shininess comparison



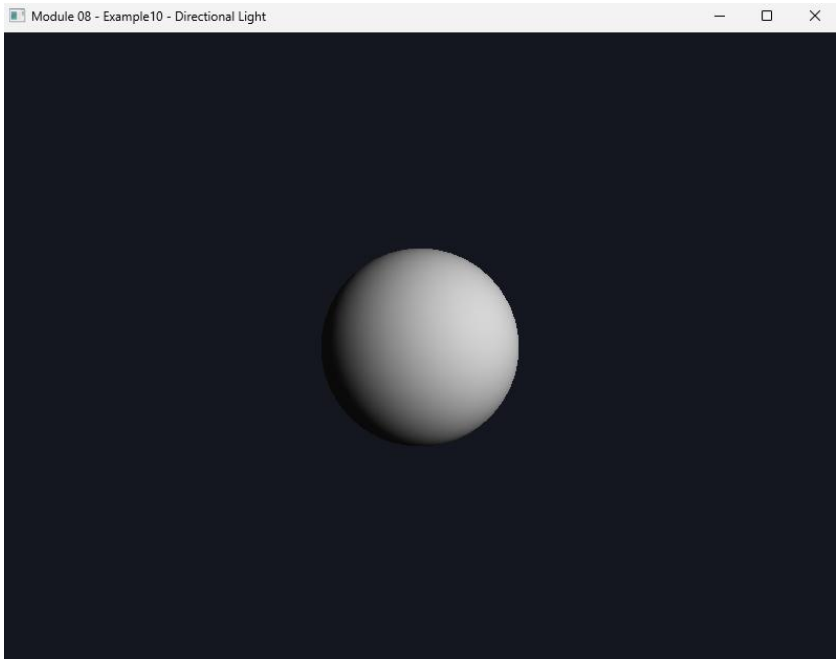
Example 08 - Raising shininess to 96 concentrates the specular reflection into a smaller, sharper highlight. This makes the material look glossier than one with low shininess.

Ex09 — Emission color



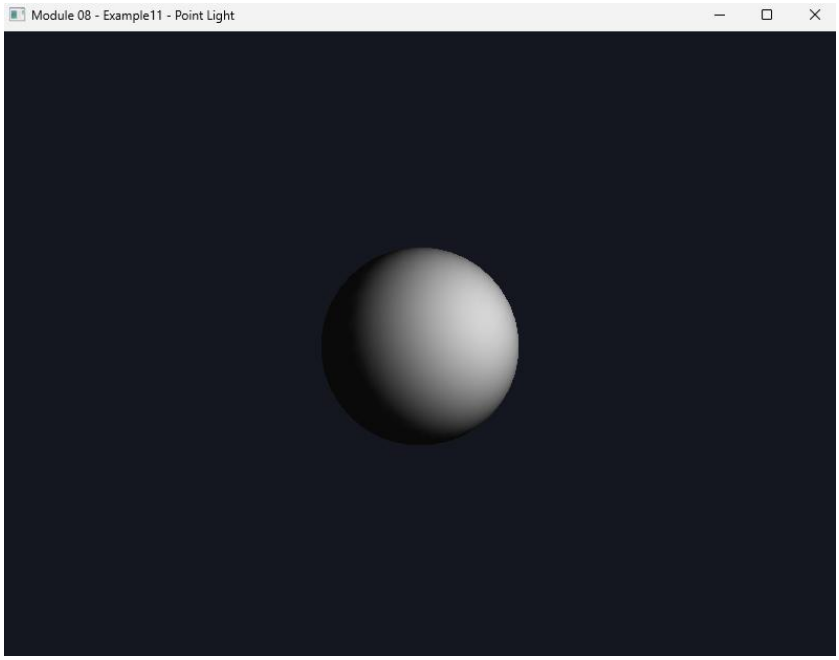
Example 09 - The orange emission property makes the sphere appear to produce some of its own color. It looks brighter, but the emission does not cast light onto other objects.

Ex10 — Directional light



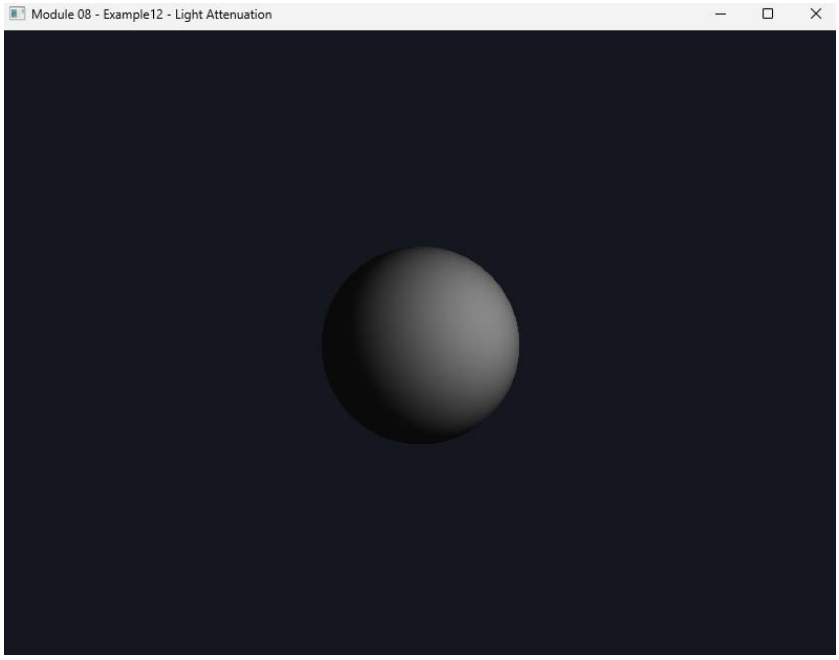
Example 10 - Setting the light position's w value to zero makes it directional, as though the rays come from a very distant source. The light direction stays consistent across the surface.

Ex11 — Point light



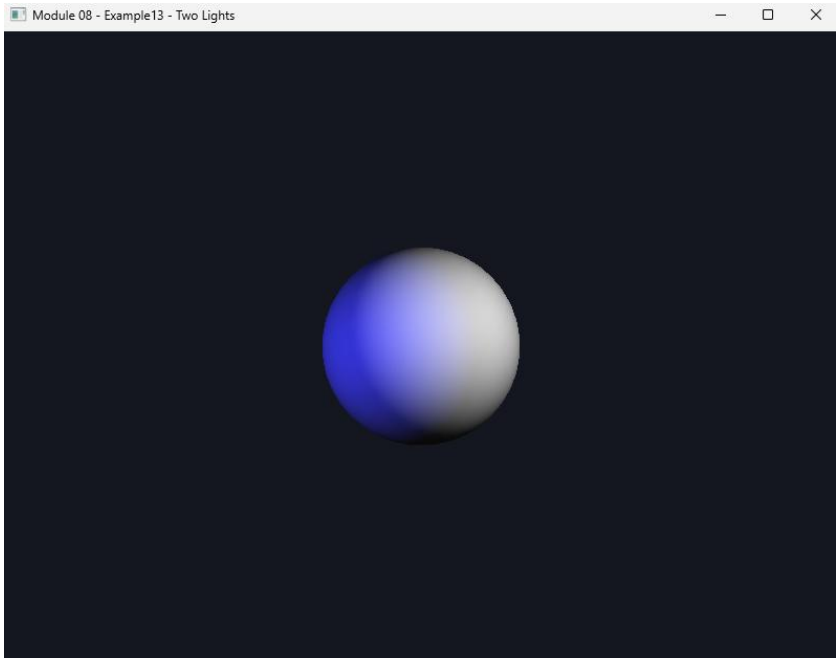
Example 11 - Setting w to one creates a positional point light. The direction from the light to the surface now varies according to the light's location.

Ex12 — Light attenuation



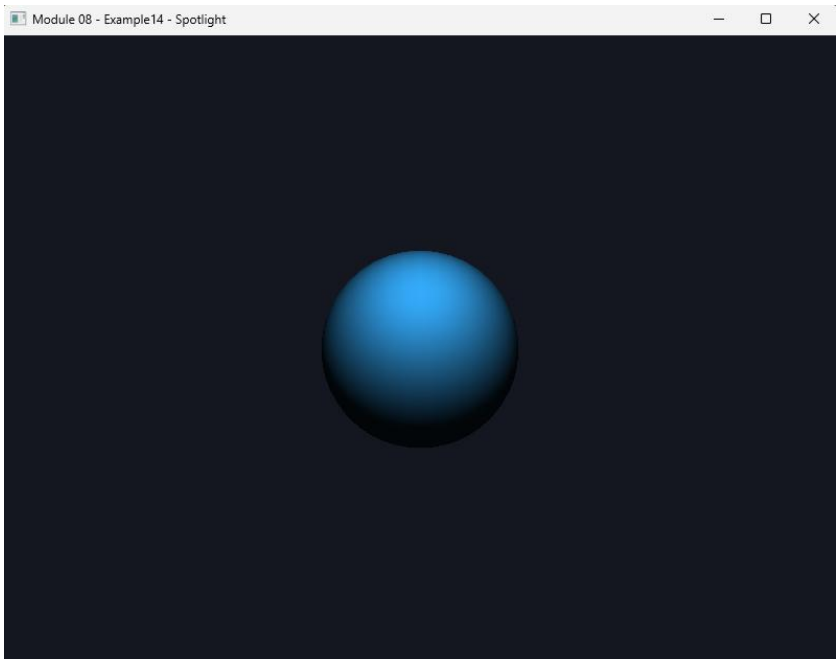
Example 12 - Linear attenuation causes the point light to weaken with distance. Surfaces farther from the source receive less illumination.

Ex13 — Two lights



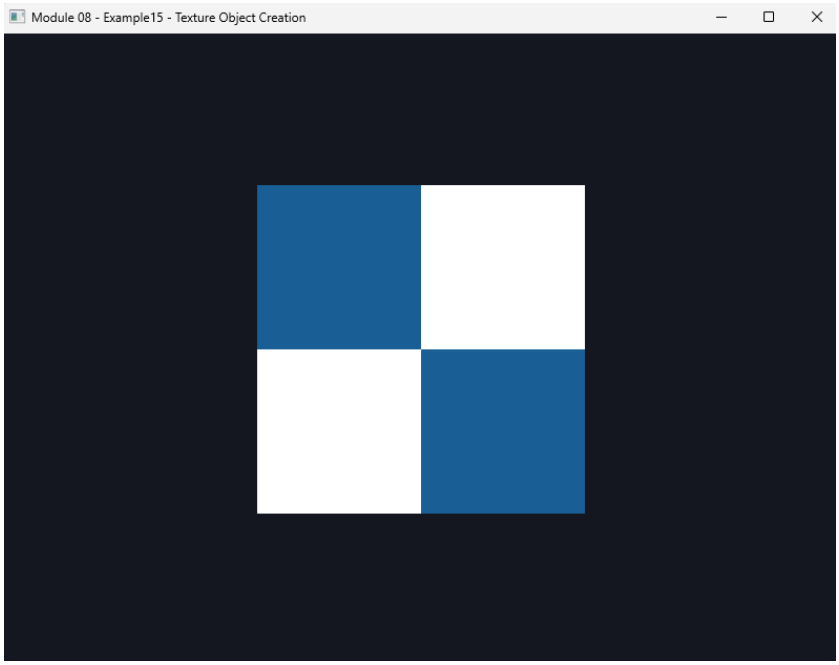
Example 13 - Enabling a second blue light adds another colored contribution from the opposite side. The sphere displays a mixture of the two light sources.

Ex14 — Spotlight



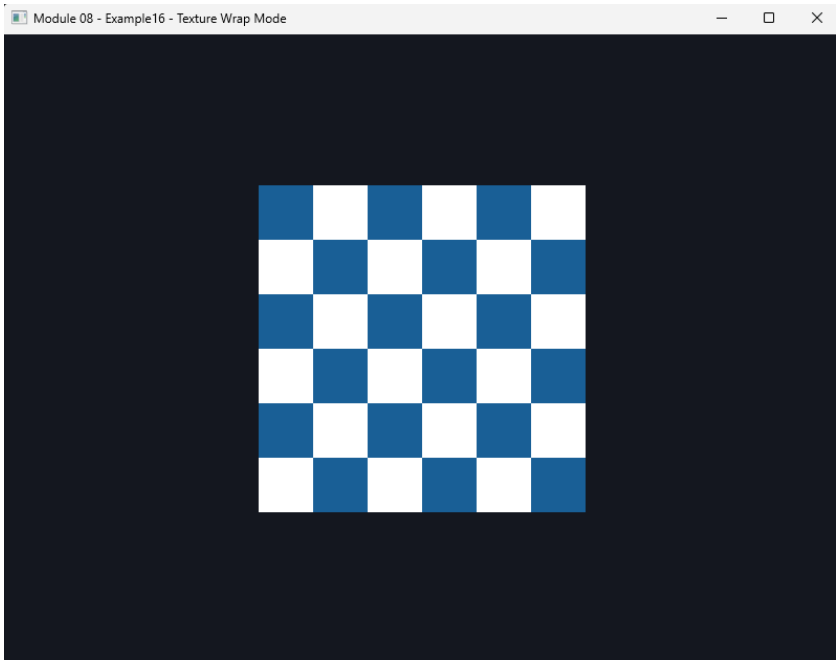
Example 14 - The point light is restricted to a cone, producing a focused spotlight effect. Areas outside the cone receive much less direct illumination.

Ex15 — Texture object creation



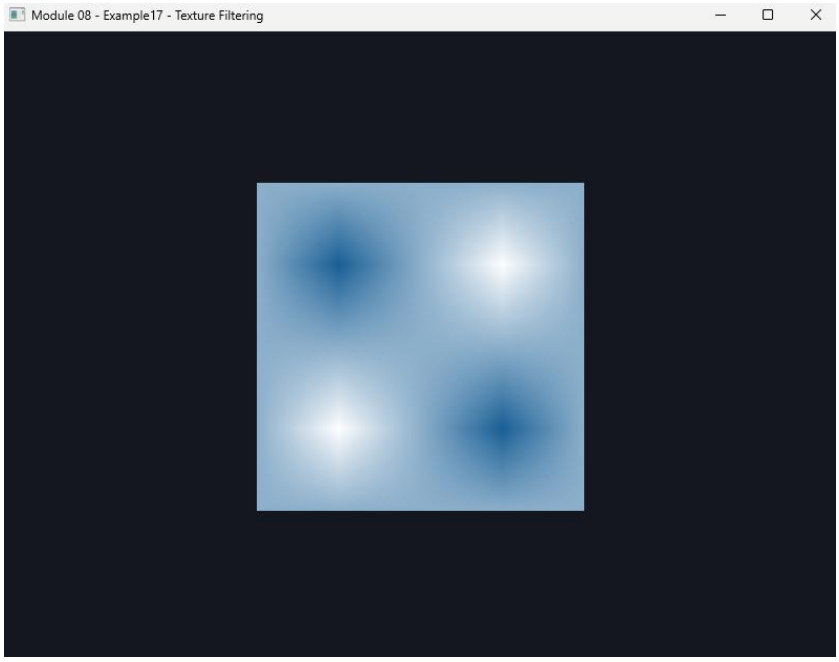
Example 15 - OpenGL allocates and binds a texture object containing a small, generated color pattern. The texture is then displayed on the quad without loading an external image.

Ex16 — Texture wrap mode



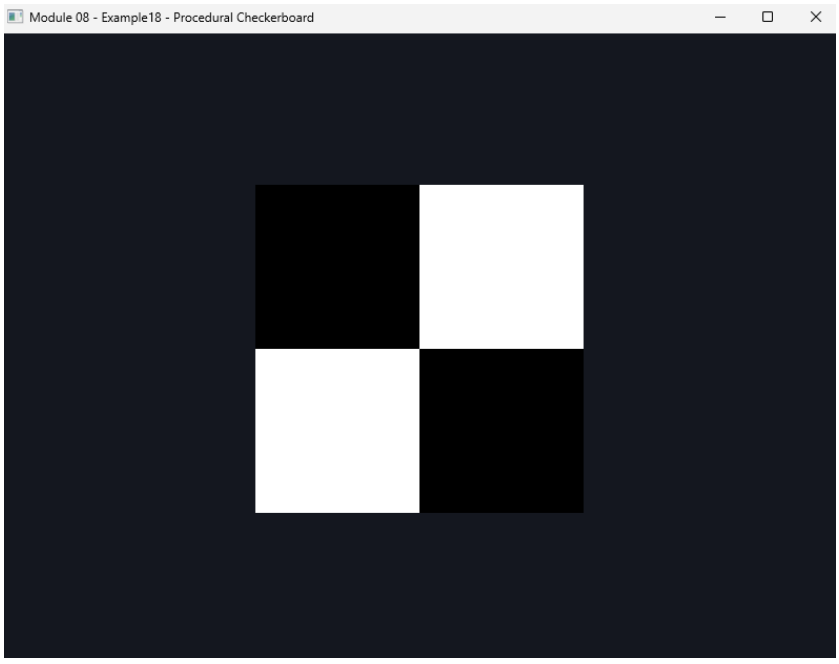
Example 16 - GL_REPEAT allows texture coordinates beyond the normal zero-to-one range to tile the image. The checker pattern repeats several times across the quad.

Ex17 — Texture filtering



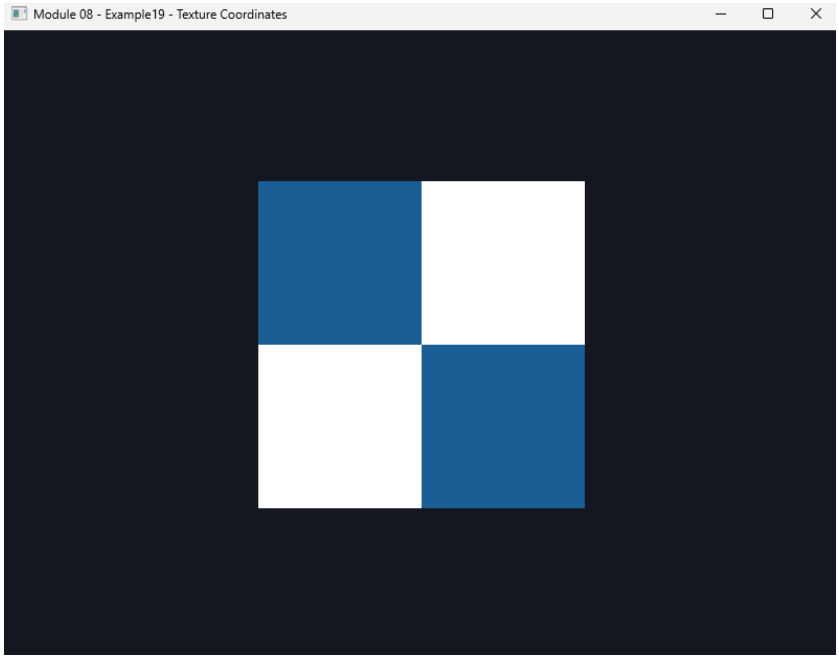
Example 17 - Linear filtering blends neighboring texels when the texture is enlarged or reduced. The checker edges appear smoother than with nearest-neighbor filtering.

Ex18 — Procedural checkerboard



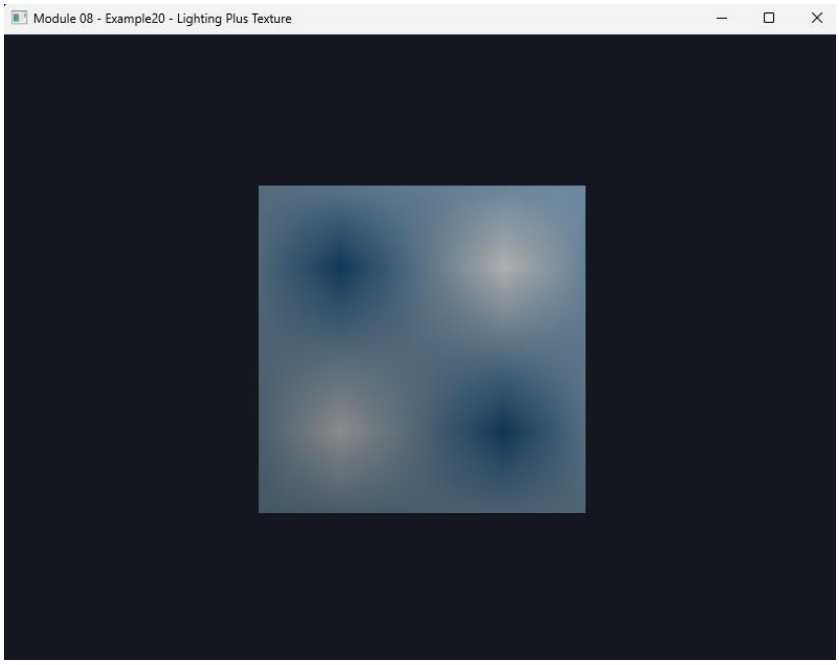
Example 18 - A checkerboard is created directly from a small RGB byte array and uploaded to OpenGL. No external image file is needed to produce the visible pattern.

Ex19 — Texture coordinates



Example 19 - Texture coordinates assign each corner of the texture to the matching corner of the quad. The complete checker image is mapped once across the surface.

Ex20 — Lighting plus texture



Example 20 - GL_MODULATE combines the checker texture with the calculated lighting. The texture remains visible while its brightness changes according to the surface illumination.

Image texture loading: complete reference pattern

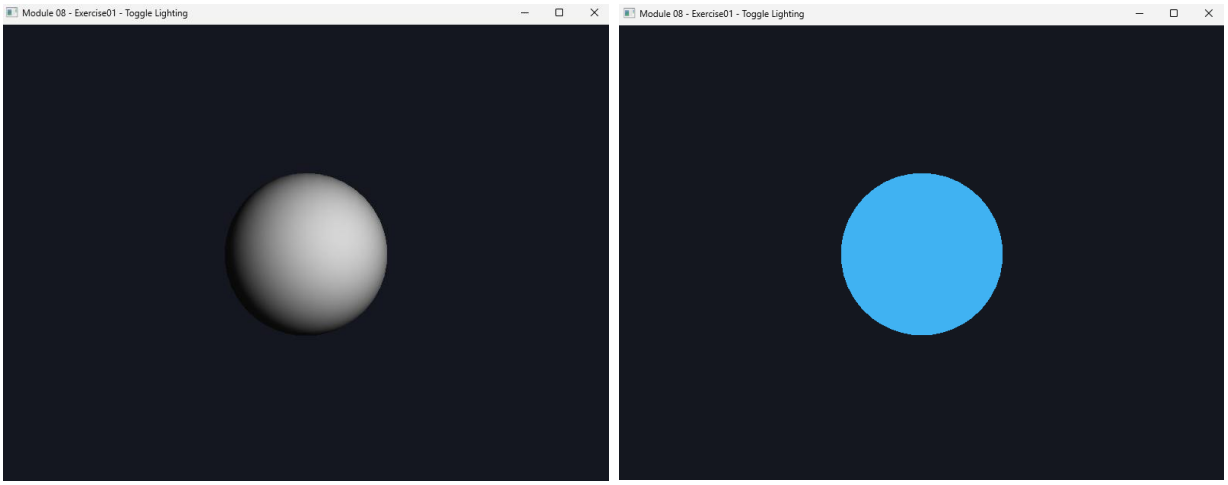
For Ex15–Ex20 and the texture exercises, this helper shows the lecture's stb_image.h → glTexImage2D workflow. Call loadTexture() once from init() after creating the OpenGL window, then glBindTexture(GL_TEXTURE_2D, textureId) before rendering textured geometry.

```
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"  // put these two lines in ONE source file
GLuint loadTexture(const char* path) {
    int w,h,n;
    stbi_set_flip_vertically_on_load(1);
    unsigned char* data = stbi_load(path,&w,&h,&n,0);
    if (!data) { std::fprintf(stderr,"Image load failed: %s\n",path); return 0; }
    GLenum fmt = (n==4) ? GL_RGBA : (n==3 ? GL_RGB : GL_RED);
    GLuint id=0; glGenTextures(1,&id); glBindTexture(GL_TEXTURE_2D,id);
    glPixelStorei(GL_UNPACK_ALIGNMENT,1);
    glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_WRAP_S,GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_WRAP_T,GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_MIN_FILTER,GL_LINEAR_MIPMAP_LINEAR);
    glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_MAG_FILTER,GL_LINEAR);
    glTexImage2D(GL_TEXTURE_2D,0,fmt,w,h,0,fmt,GL_UNSIGNED_BYTE,data);
    glGenerateMipmap(GL_TEXTURE_2D);
    stbi_image_free(data); return id;
}
```

PART B — PRACTICE EXERCISES

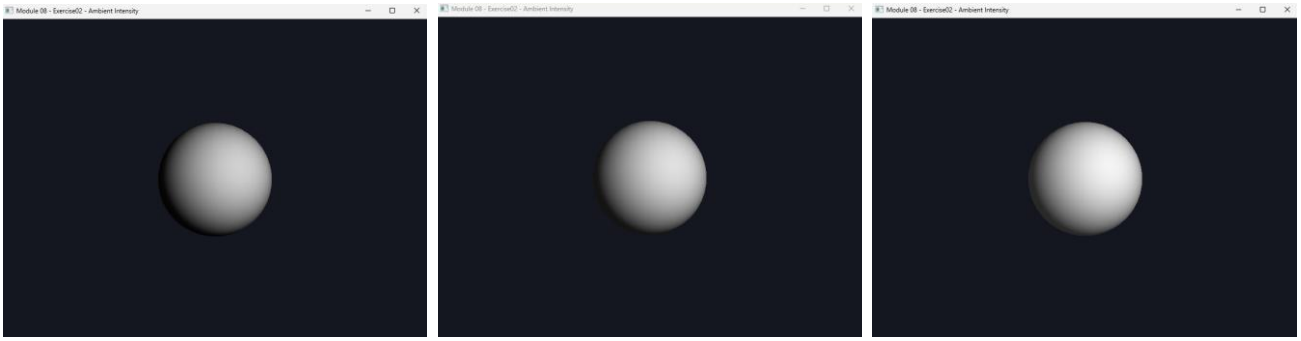
Complete all tasks independently. Easy Q01–Q07; Medium Q08–Q14; Hard Q15–Q20. Each task requires a .cpp file, output screenshot, and a short explanation.

Q01 [Easy] — Toggle lighting with the L key



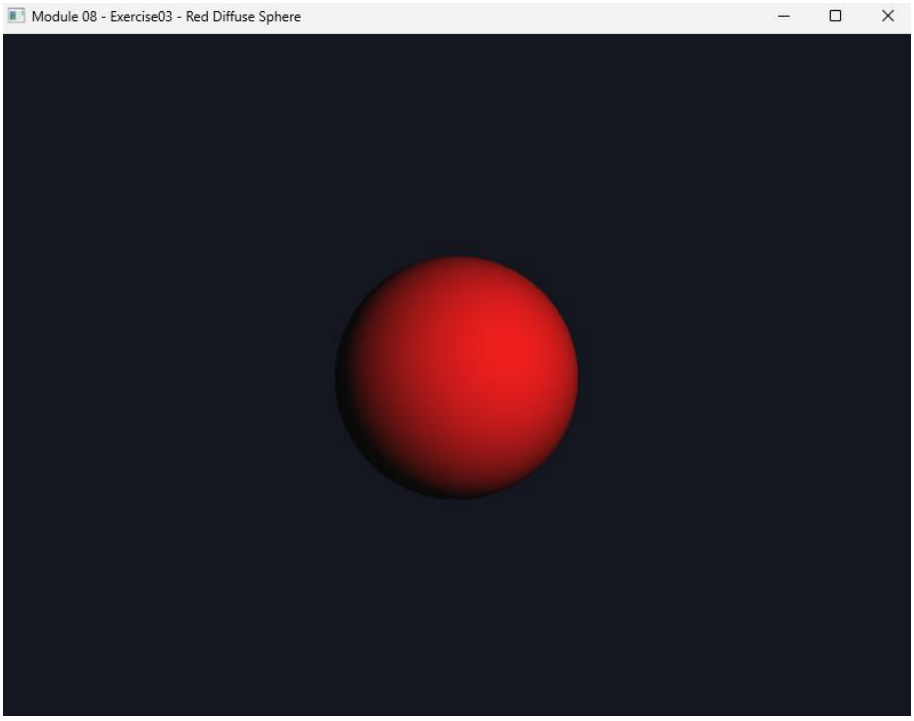
Exercise 01 - Pressing L switches fixed-function lighting on and off. With lighting enabled, the sphere has gradual shading and visible depth; without it, the current color is applied uniformly and the sphere looks flatter.

Q02 [Easy] — Change ambient intensity



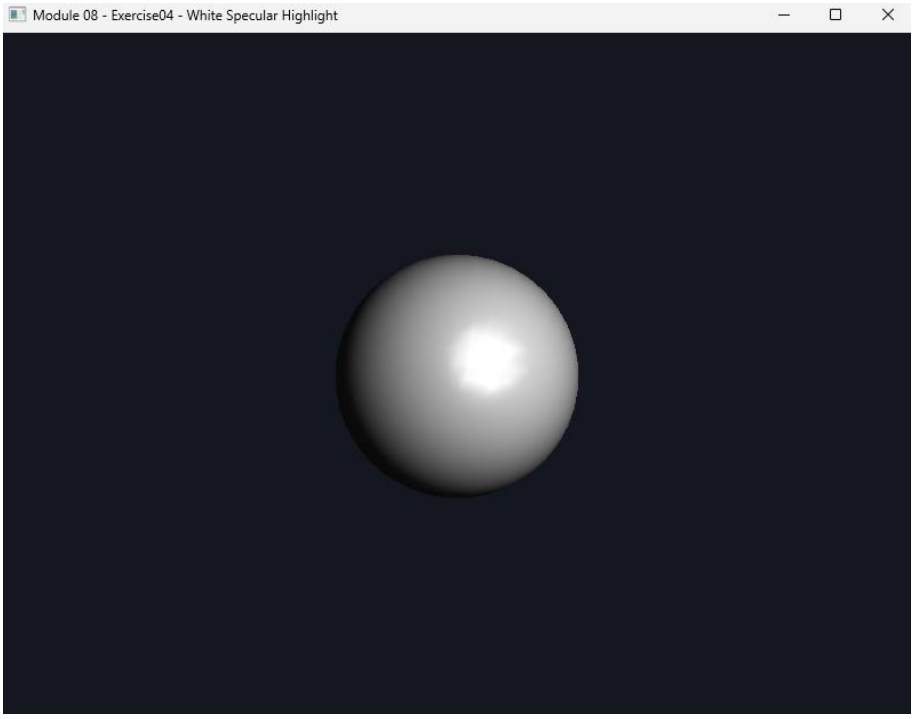
Exercise 02 - Keys 1, 2, and 3 select global ambient intensities of 0.1, 0.4, and 0.8. Higher values brighten the shadowed side most strongly because ambient light affects the entire surface.

Q03 [Easy] — Red diffuse sphere



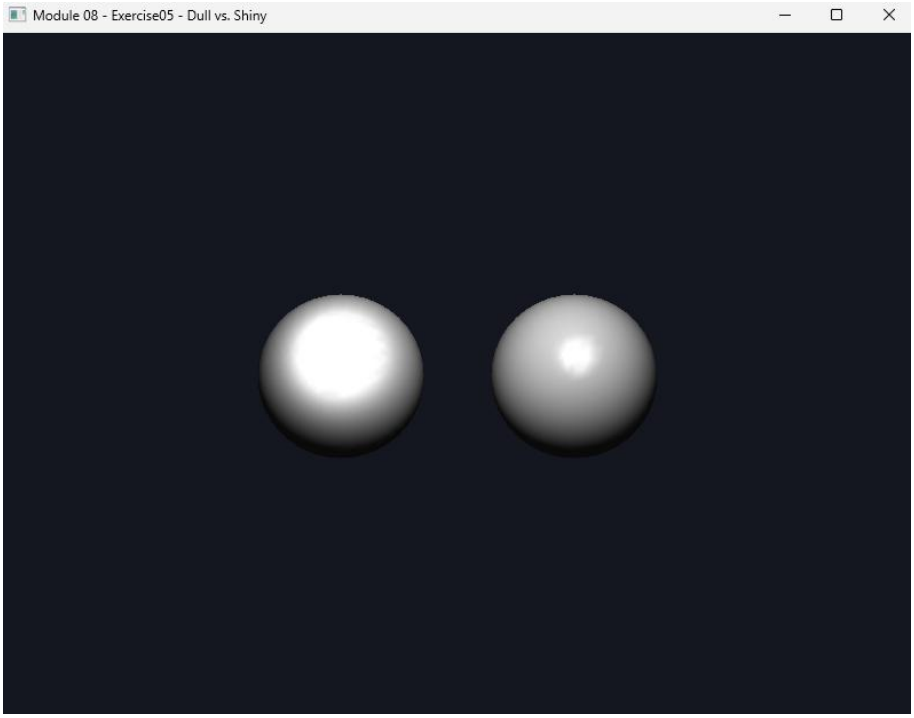
Exercise 03 - The sphere's diffuse material is set to red, so directly illuminated areas appear bright red. Regions turned away from the light remain darker because diffuse reflection depends on the surface normal.

Q04 [Easy] — White specular highlights



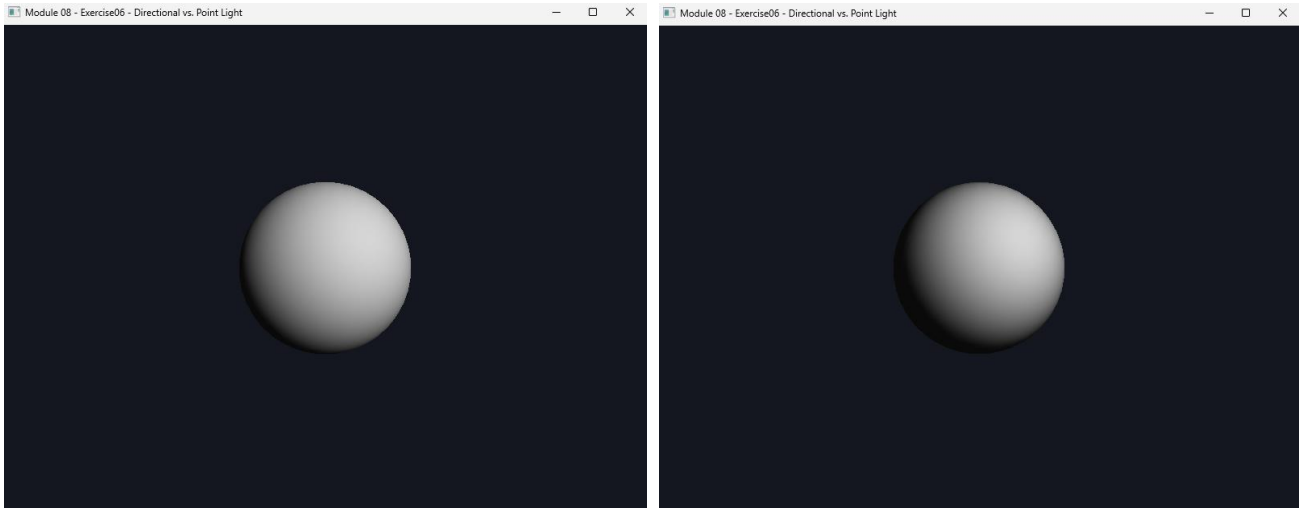
Exercise 04 - White specular properties on both the light and material create a bright reflected highlight. The highlight appears on the part of the sphere whose normal best reflects the light toward the camera.

Q05 [Easy] — Dull versus shiny



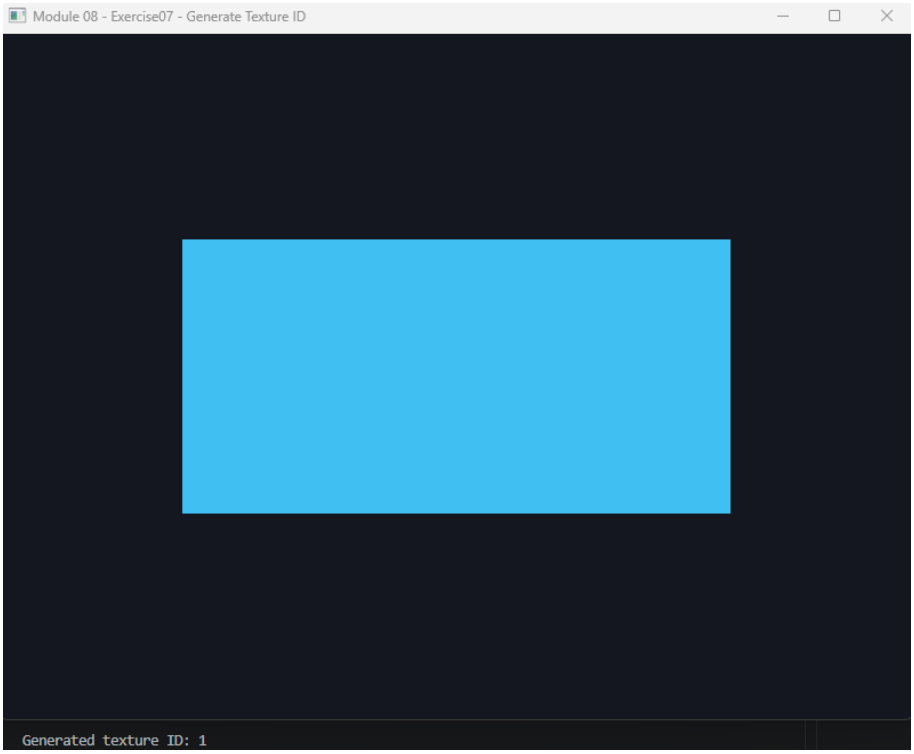
Exercise 05 - The left sphere uses shininess 8, producing a broad and soft highlight. The right sphere uses shininess 96, creating a much smaller, sharper highlight that looks more polished.

Q06 [Easy] — Directional versus point light



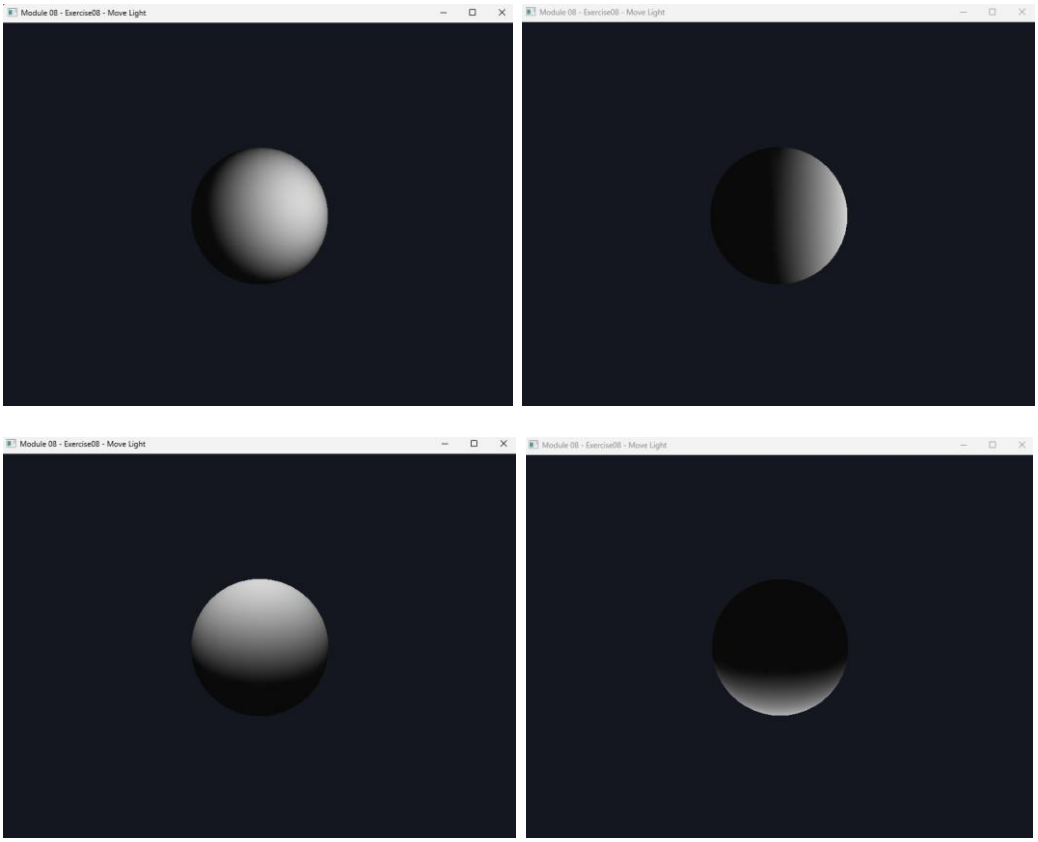
Exercise 06 - Pressing P changes the light between directional mode with w=0 and positional mode with w=1. A directional source uses consistent parallel rays, while a point source illuminates according to its location relative to the sphere.

Q07 [Easy] — Generate a texture ID



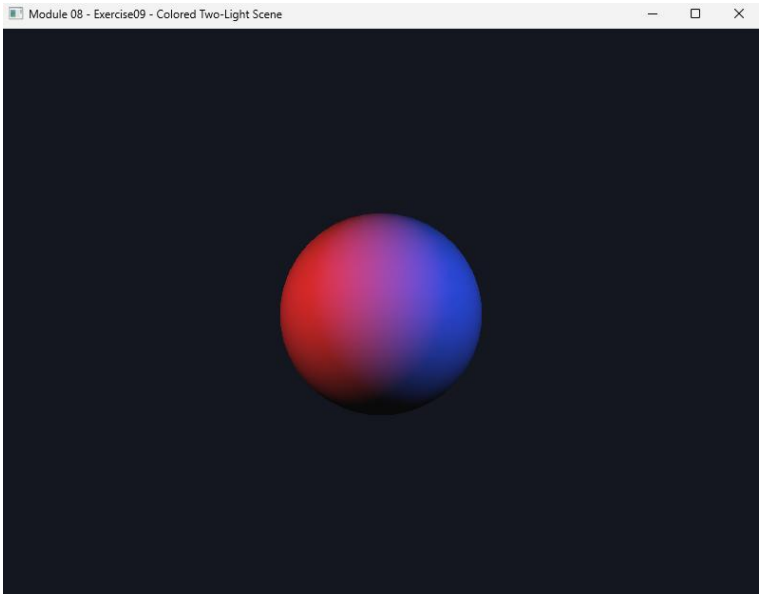
Exercise 07 - OpenGL generates and binds a nonzero texture ID, which is printed in the console. Pressing ESC deletes the texture before the program exits, demonstrating the complete object lifecycle.

Q08 [Medium] — Move a light with arrow keys



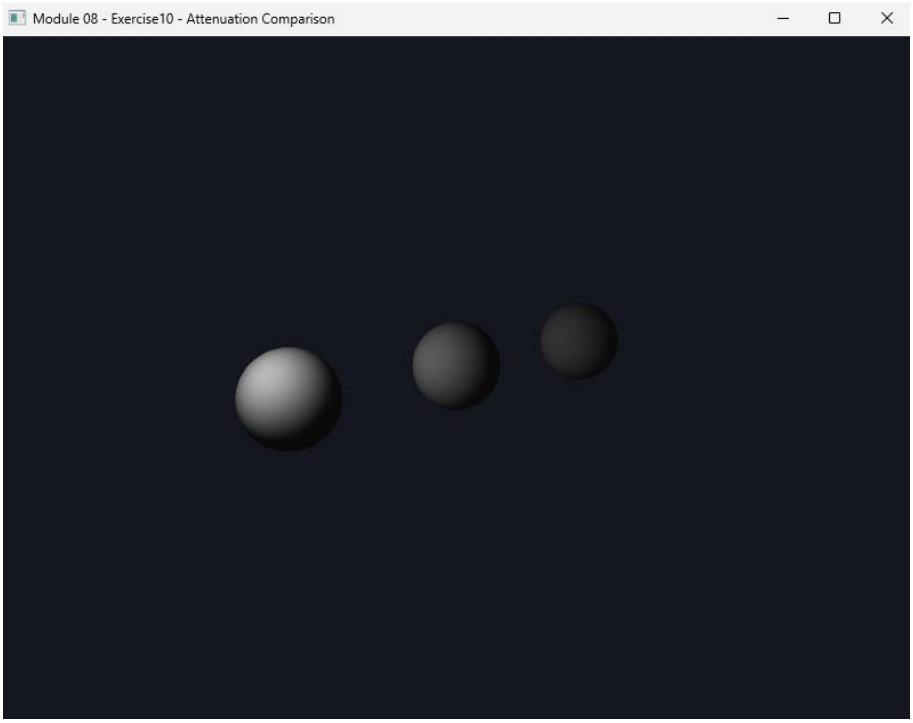
Exercise 08 - The arrow keys move the point light horizontally and vertically. As its position changes, the brightest region and highlight travel across the sphere, showing that lighting is recalculated from the new coordinates.

Q09 [Medium] — Colored two-light scene



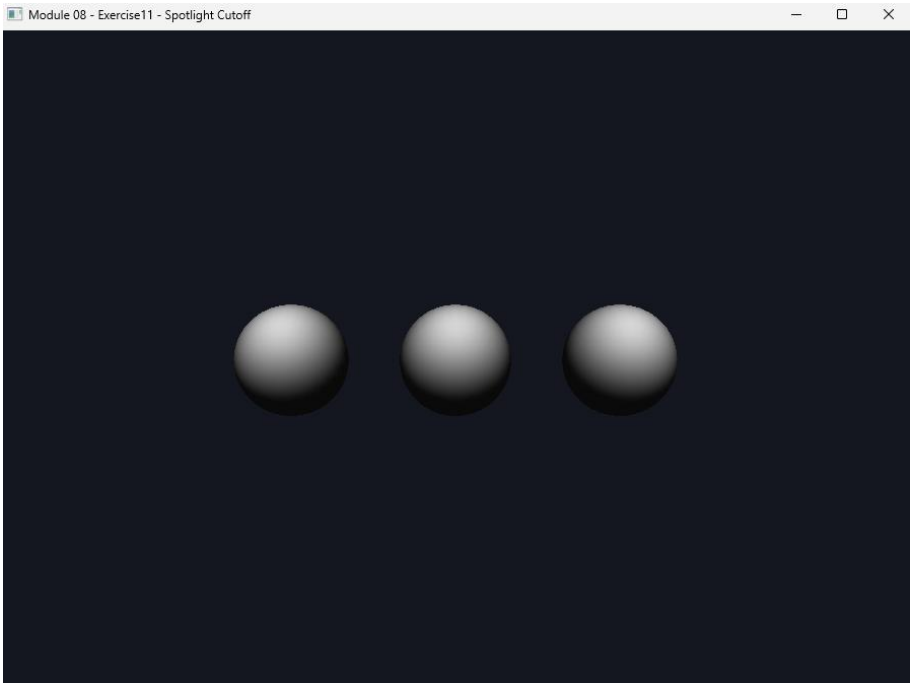
Exercise 09 - Red light from the left and blue light from the right illuminate the same sphere. Their contributions overlap to form mixed purple tones, while areas closer to one source favor that source's color.

Q10 [Medium] — Attenuation comparison



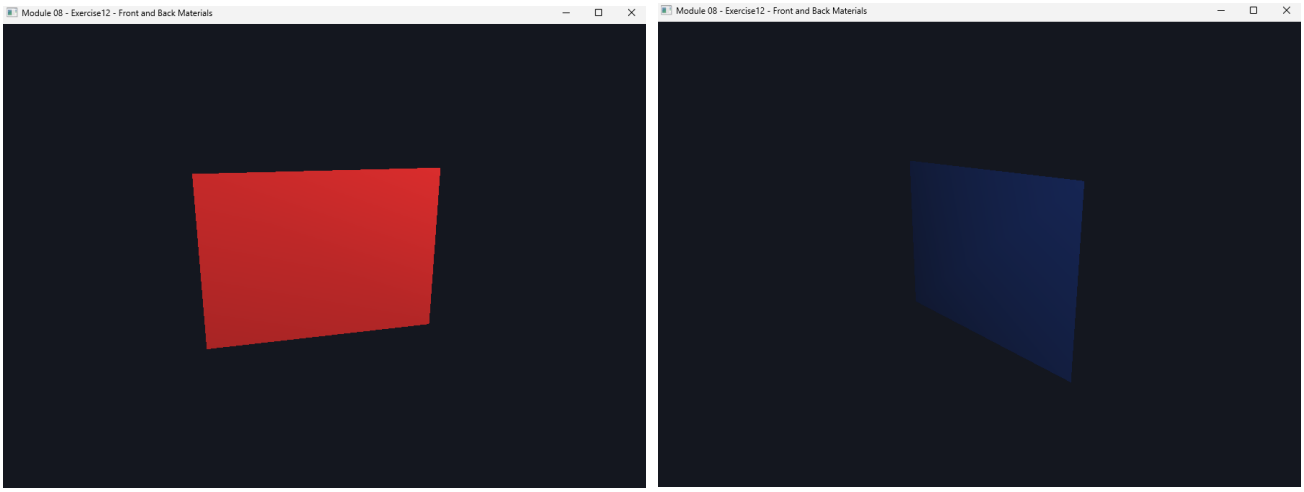
Exercise 10 - Three spheres are placed at increasing distances from an attenuated point light. The closest sphere is brightest, while the farther spheres receive progressively weaker illumination.

Q11 [Medium] — Spotlight cutoff



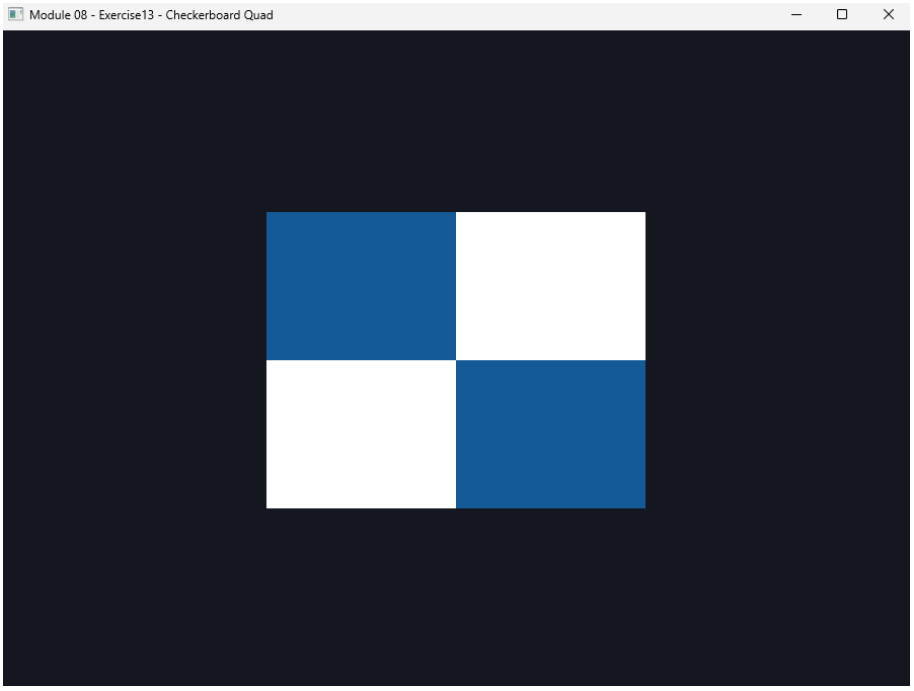
Exercise 11 - The three spotlights use cutoffs of 15, 30, and 60 degrees. A small cutoff creates a narrow focused cone, while a larger cutoff spreads illumination over a wider area.

Q12 [Medium] — Front and back materials



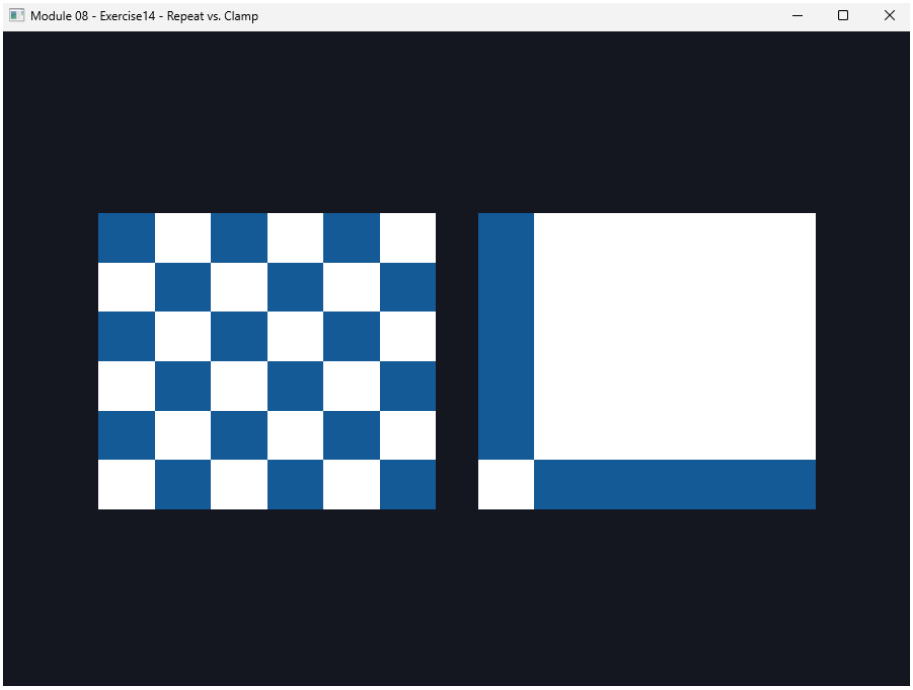
Exercise 12 - The polygon has a red front material and blue back material and rotates continuously. As it turns, OpenGL selects the material associated with the face currently visible to the camera.

Q13 [Medium] — Checkerboard quad



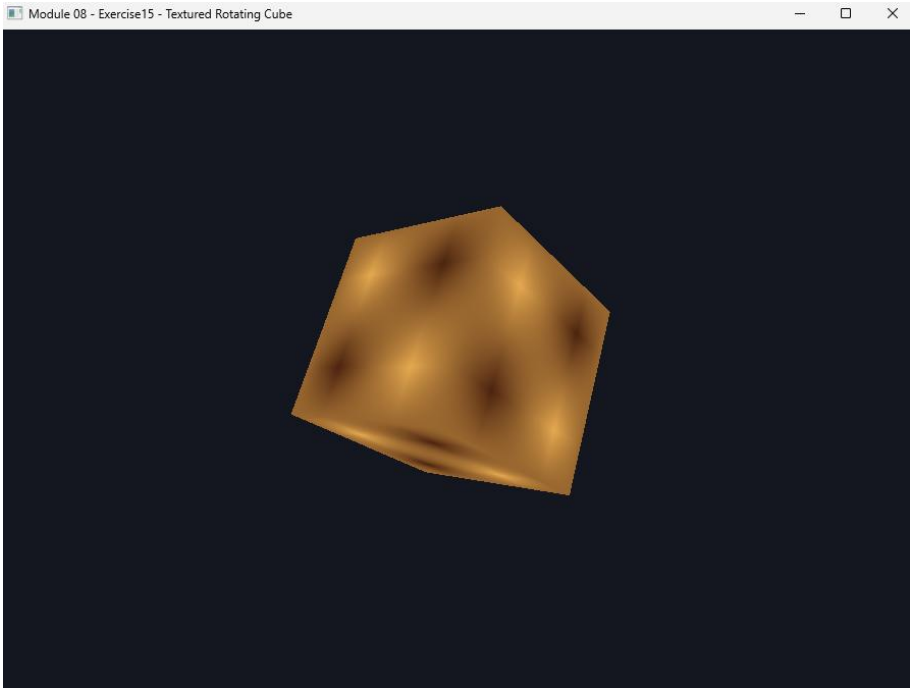
Exercise 13 - A procedural two-by-two RGB checkerboard is uploaded and mapped across a quad. The visible pattern is produced entirely from an in-memory byte array, so no image file is required.

Q14 [Medium] — Texture repeat versus clamp



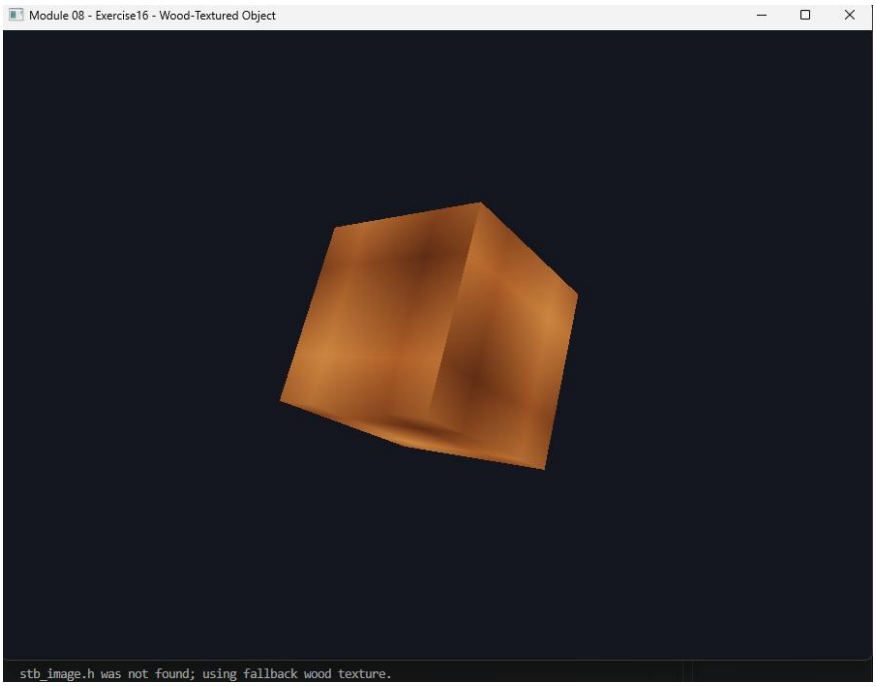
Exercise 14 - The left quad uses GL_REPEAT, causing coordinates from zero to three to tile the checkerboard. The right quad uses GL_CLAMP_TO_EDGE, so the border texels stretch across coordinates outside the normal range.

Q15 [Hard] — Textured rotating cube



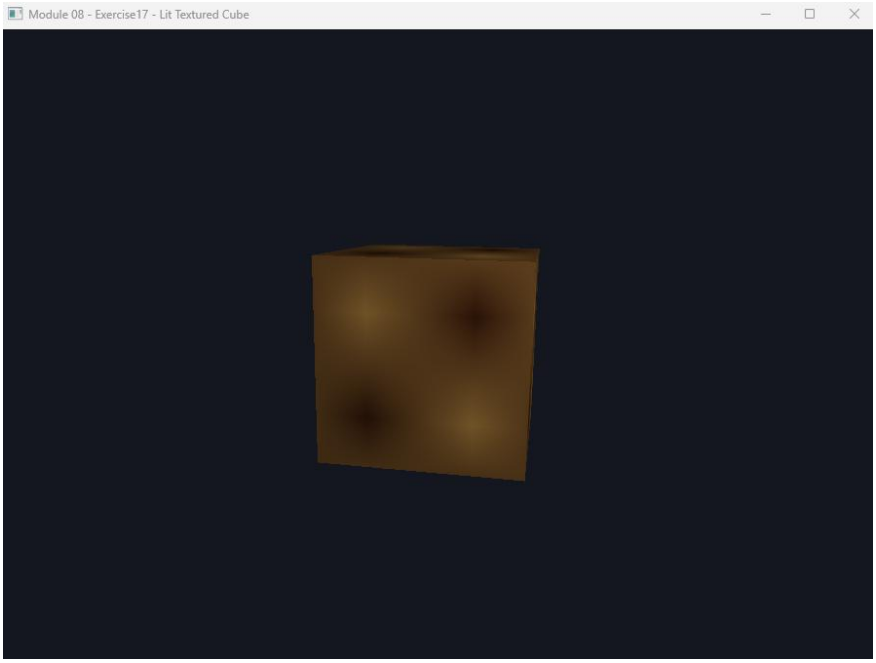
Exercise 15 - A generated texture is mapped to every face of a cube while the cube rotates around two axes. The changing orientation shows that each face has its own texture coordinates and remains correctly mapped during animation.

Q16 [Hard] — Wood-textured object



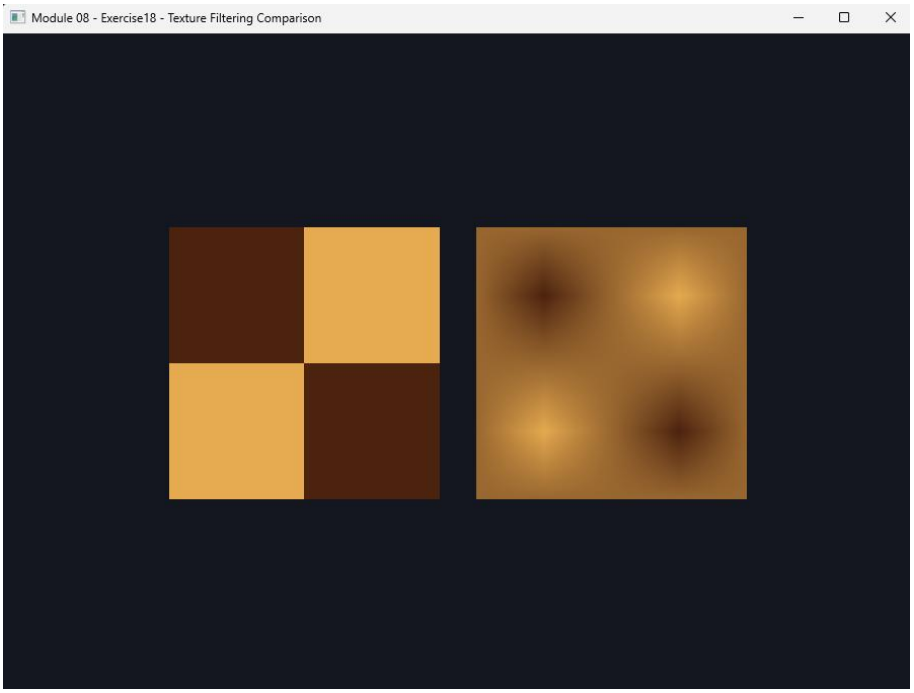
Exercise 16 - The program attempts to load woodtexture.jpg through stb_image when the header and image are available. If either is missing or loading fails, it reports the problem and uses a generated wood-colored pattern so the cube still renders safely.

Q17 [Hard] — Lit textured cube



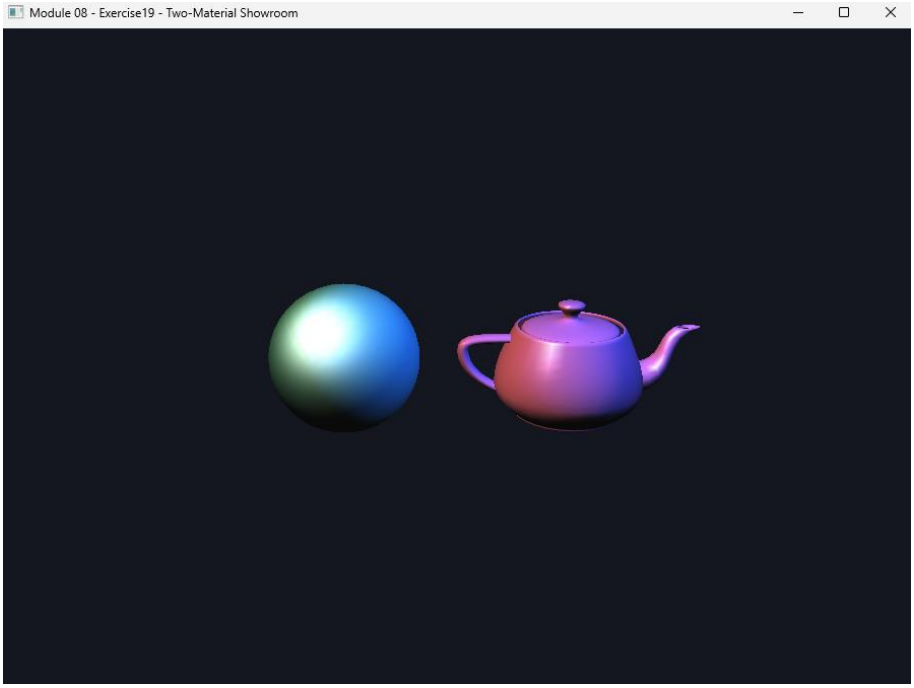
Exercise 17 - The texture color is modulated by fixed-function lighting on a rotating cube. Correct normal vectors are essential because they tell OpenGL how each face is oriented relative to the light, producing believable bright and dark faces.

Q18 [Hard] — Texture filtering comparison



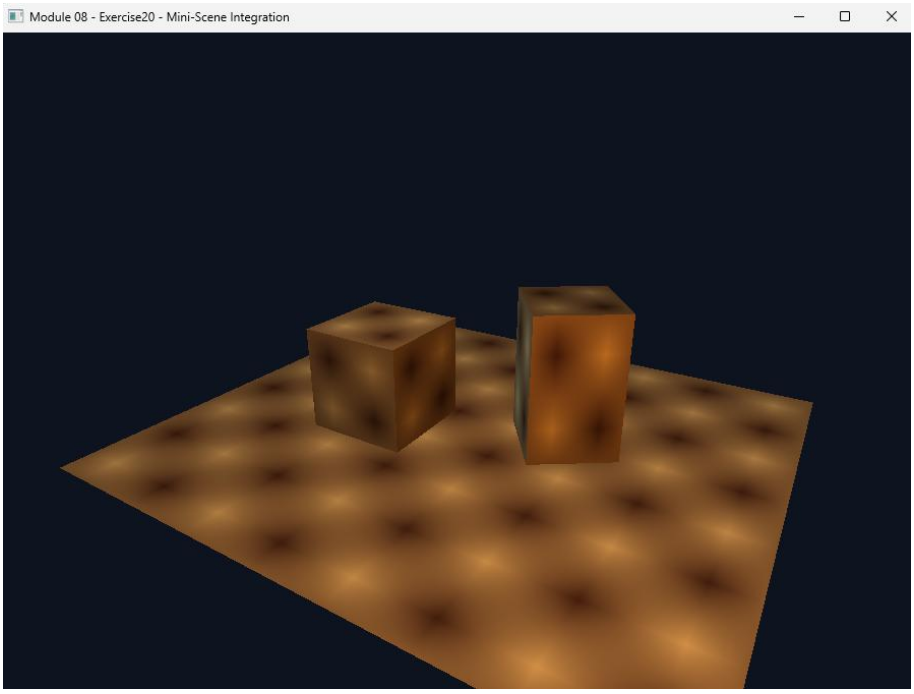
Exercise 18 - The left enlarged texture uses GL_NEAREST and therefore shows hard, pixelated texel boundaries. The right texture uses GL_LINEAR, which blends neighboring texels to create smoother transitions.

Q19 [Hard] — Two-material showroom



Exercise 19 - A dull sphere and a shiny teapot are displayed under warm and cool lights. The dull object has a broad weak reflection, while the high-shininess object produces tighter highlights that also carry the combined light colors.

Q20 [Hard] — Mini-scene integration



Exercise 20 - The mini-scene combines a repeating textured floor with two textured 3D objects. A positional warm light and a cool directional light interact with the supplied normals, texture modulation, and depth testing to produce an integrated scene.